MANUAL DE DESPLIEGUE Y OPERACIÓN

SH-VT-TEC-001 | Versión 1.0

SECUREHUB / VULNTRACK

PLATAFORMA DE GESTIÓN DE VULNERABILIDADES Y RIESGOS

MANUAL DE DESPLIEGUE Y OPERACIÓN

Título del proyecto	SecureHub / VulnTrack
Subtitulo	**Plataforma de Gestión de Vulnerabilidades y Riesgos**
Documento	**Manual de Despliegue y Operación**
Equipo de Ingeniería	**Muñoz Rey Hilary Michelle** *Analista de Documentación Técnica* **Ruiz Camelo Jennifer Dayan** *Especialista en Documentación de Software* **Molina Lancheros Daniel Steven** *Ingeniero DevSecOps* **Torres Pérez Eduar De Jesús** *Desarrollador de Software* **Alexander Diaz Molano** *Analista Técnico de Arquitectura*
Evaluador Técnico	Ospina Navas Jaider *Magíster en Ciberseguridad y Ciberdefensa*
Fecha de entrega	23 de octubre de 2026
Licenciamiento	MIT License (Código abierto) *Copyright © 2026 Equipo VulnTrack*
Versión del documento	**V1.0 — revisión de evidencias** *(10/10/2026)*

1. Objetivo y alcance

Establecer el procedimiento para instalar, desplegar, verificar, operar y detener SecureHub/VulnTrack en un equipo Windows mediante Docker Desktop y Docker Compose. La guía incluye configuración de variables, servicios, monitoreo, operación cotidiana, respaldo de base de datos y diagnóstico.

2. Arquitectura desplegada

Servicio	Función	Exposición/puerto observado en Compose
securehub-frontend	SPA React compilada y servida por Nginx	80 → 8080
securehub-api	API Node.js/Express	127.0.0.1:3000 → 3000
securehub-db	PostgreSQL 18	5432 solo en red interna
securehub-redis	Redis 7 con autenticación	6379 solo en red interna
securehub-worker	Procesos worker, SLA/NVD	Red interna
securehub-scanner	Escaneo del repositorio con Trivy	Red interna
securehub-files	Nginx para evidencias estáticas	Red interna
prometheus	Métricas y alertas	9090
alertmanager	Gestión de alertas	9093
grafana	Visualización de métricas	3001 → 3000
loki / alloy	Agregación y envío de logs	Loki interno; Alloy interno

3. Requisitos previos

- Equipo con Windows 10/11, PowerShell y Git para Windows instalados.
- Docker Desktop instalado, iniciado y configurado para utilizar el motor Linux; Docker Compose v2 disponible.
- Acceso al repositorio y conectividad para descargar imágenes.
- Espacio disponible para descargar imágenes y almacenar datos persistentes de PostgreSQL, Redis, evidencias, logs, caché de Trivy y métricas. La demanda de recursos depende del volumen de datos y del tiempo de operación.
- Puertos locales libres: 80, 3000, 3001, 9090 y 9093. El puerto 5173 solo se usa si Vite se ejecuta aparte.

```
git --version
docker --version
docker compose version
```

4. Clonar y preparar el proyecto

```
git clone https://github.com/3D74R08/VulnTrack.git
cd .\VulnTrack
```

Confirme que el directorio contiene `docker-compose.yml`, `.env.example` y las carpetas del proyecto, incluidas `backend`, `frontend`, `base_datos`, `config`, `deploy`, `monitoring`, `scanner` y `security`.

5. Configurar variables de entorno

```
Copy-Item .env.example .env
notepad .env
```

Edite `.env` y complete las variables del archivo `.env.example`: `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_APP_PASSWORD`, `REDIS_PASSWORD`, `CORS_ORIGINS` y `JWT_SECRET`. Configure también, según el uso, `DEMO_ADMIN_ENABLED`, `DEMO_ADMIN_NAME`, `DEMO_ADMIN_EMAIL`, `DEMO_ADMIN_PASSWORD`, `NVD_API_KEY` y los parámetros `SMTP_*`. Genere secretos robustos y no reutilice contraseñas personales. Mantenga `DEMO_ADMIN_ENABLED=false` salvo que necesite crear una cuenta de demostración en un entorno local nuevo.

Variable/grupo	Uso	Recomendación
DB_NAME, DB_USER, DB_PASSWORD	Base y usuario administrativo de PostgreSQL	Valores locales únicos; no reutilizar credenciales de producción.
DB_APP_PASSWORD	Credencial de app_user, cuenta de menor privilegio	Generar una contraseña fuerte independiente.
REDIS_PASSWORD	Protección de Redis	Definir contraseña no vacía y no compartirla.
JWT_SECRET	Firma de tokens de autenticación	Secreto aleatorio largo; rotar si se expone.
CORS_ORIGINS	Lista de orígenes permitidos	Limitar a los orígenes usados; no permitir cualquier origen.
DEMO_ADMIN_*	Cuenta local de demostración	Activar solo para bootstrap de entorno local y desactivar después.
NVD_API_KEY	Consulta a NVD	Opcional según el uso y límites de la fuente.
SMTP_*	Notificaciones por correo	Configurar solo si se probará el envío de correo.
SCANNER_INTERVAL_MS	Intervalo del escáner	Usar un valor apropiado para pruebas locales.

Protección: No adjuntar .env a informes, capturas ni repositorios. Si se incluyó en un ZIP compartido o se sospecha exposición, rotar los secretos.

6. Validar la configuración

```
docker compose config --quiet
```

Si el comando devuelve un error, corregir variables ausentes, sintaxis YAML o referencias a archivos antes de continuar. No seguir al despliegue hasta que la configuración sea válida.

7. Construir e iniciar el despliegue completo

```
docker compose up -d --build
```

Docker descargará imágenes y construirá las imágenes propias de API, frontend, worker y scanner. La primera ejecución puede tardar por la descarga de capas. En la evidencia entregada, las imágenes se construyeron y varios servicios quedaron Up/healthy.

8. Verificar el despliegue

1. Consultar el estado de los servicios.
2. Revisar los logs de servicios que no aparezcan healthy o que reinicien.
3. Abrir la aplicación en el navegador.
4. Comprobar el endpoint de salud de la API.
5. Verificar que el inicio de sesión y una operación funcional de prueba trabajen correctamente.

```
docker compose ps
docker compose logs --tail=100
docker compose logs --tail=100 securehub-api
docker compose logs --tail=100 securehub-db
```

URL	Uso
http://localhost/	Frontend contenedorizado, puerto 80
http://localhost:3000/health	Endpoint de salud de la API
http://localhost:9090/	Prometheus, si está habilitado
http://localhost:9093/	Alertmanager, si está habilitado
http://localhost:3001/	Grafana, si está habilitado
http://localhost:5173/	Frontend Vite, solo si se inicia manualmente

9. Inicialización y persistencia de PostgreSQL

Compose monta scripts SQL en /docker-entrypoint-initdb.d. PostgreSQL ejecuta estos scripts únicamente cuando inicializa un volumen de datos vacío. El servicio securehub-db-bootstrap ejecuta tareas idempotentes de configuración/migración definidas en Compose, incluidas semillas de roles y migraciones seleccionadas.

Precaución: No eliminar volúmenes ni restaurar respaldos sobre datos que deban conservarse. docker compose down conserva volúmenes por defecto; docker compose down -v puede eliminar datos persistentes.

10. Operación diaria

Tarea	Comando
Ver estado	docker compose ps
Ver logs recientes	docker compose logs --tail=100
Seguir logs de API	docker compose logs -f securehub-api
Reiniciar un servicio	docker compose restart securehub-api
Detener contenedores	docker compose stop
Iniciar contenedores detenidos	docker compose start
Reconstruir tras cambios	docker compose up -d --build
Eliminar contenedores/red creada	docker compose down

11. Copias de seguridad y restauración

El despliegue reserva el volumen `backups-data` en `/var/backups`. La copia de seguridad descrita a continuación se ejecuta manualmente y debe almacenarse también fuera del equipo o del volumen operativo, en una ubicación protegida.

```
docker compose exec -T securehub-db pg_dump -U <usuario_db> -d <nombre_db> -Fc > .\securehub-backup.dump
```

Sustituya los marcadores por los valores de `DB_USER` y `DB_NAME` definidos en `.env`. Guarde el archivo en una ubicación protegida y compruebe periódicamente la restauración en un entorno de prueba.

12. Solución de problemas

Síntoma	Diagnóstico y acción
docker compose config falla	Revisar .env, nombres de variables y sintaxis del YAML.
Servicio no arranca	Ejecutar docker compose ps y docker compose logs --tail=200 <servicio>.
API no saludable	Comprobar GET /health, conectividad con PostgreSQL/Redis y variables de entorno.
Frontend no abre en localhost	Confirmar que securehub-frontend está activo y que el puerto 80 está libre.
Error de permisos en volúmenes	Revisar los servicios de inicialización de volúmenes y los logs asociados.
Cambios de esquema no aparecen	Recordar que initdb.d no se ejecuta de nuevo sobre un volumen inicializado; revisar las migraciones y el bootstrap.
Puerto ocupado	Identificar el proceso local o cambiar el mapeo de forma coordinada y documentada.

La importación de imágenes tarda	Esperar a que finalicen las descargas; revisar conectividad y espacio en disco.

13. Seguridad operativa y limitaciones

- Este procedimiento describe el despliegue local de referencia. Para operarlo en una red compartida, limite el acceso a las consolas de monitoreo, configure controles de red y TLS en el punto de publicación, proteja los secretos y aplique las políticas de respaldo de la organización.
- El frontend se publica por el puerto 80; la API se publica en `127.0.0.1:3000`, por lo que su acceso directo queda restringido al equipo local.
- Prometheus y Alertmanager se publican en los puertos 9090 y 9093. En equipos compartidos, limite su acceso mediante controles de red y no comparta sus consolas con usuarios no autorizados.
- Alloy utiliza el socket de Docker para recopilar información de contenedores. Trate este acceso como privilegiado y ejecute el entorno únicamente bajo cuentas y equipos autorizados.
- Mantener imágenes y dependencias actualizadas y revisar los resultados de escaneo antes de liberar cambios.

14. Evidencia de aceptación

- Salida de docker compose config --quiet sin errores.
- Captura de docker compose ps con estado de servicios.
- Captura del frontend abierto en localhost.
- Respuesta correcta de /health.
- Logs sin errores críticos no resueltos.
- Registro de fecha, commit/versión y persona que realizó la verificación.

15. Procedimiento operativo paso a paso

Este procedimiento utiliza PowerShell y Docker Desktop en Windows. Ejecute los comandos desde la carpeta raíz del repositorio, donde se encuentra `docker-compose.yml`.

15.1 Preparar el equipo y abrir el proyecto

1. Abra Docker Desktop y espere hasta que indique que el motor está en ejecución.
2. Abra PowerShell.
3. Clone el repositorio y acceda a la carpeta del proyecto:

```
git clone https://github.com/3D74R08/VulnTrack.git
cd .\VulnTrack
```

Si el repositorio ya está descargado, abra PowerShell en esa carpeta y continúe con el siguiente paso.

15.2 Crear y proteger la configuración

Cree el archivo local de configuración a partir de la plantilla:

```
Copy-Item .env.example .env
notepad .env
```

Complete las variables que utiliza Docker Compose. Defina contraseñas robustas y un valor aleatorio para `JWT_SECRET`. Configure `CORS_ORIGINS` para el origen desde el que se abrirá la aplicación. Si se usa SMTP, complete los parámetros `SMTP_HOST`, `SMTP_PORT`, `SMTP_SECURE`, `SMTP_USER`, `SMTP_PASS` y `SMTP_FROM`. La clave `NVD_API_KEY` es opcional para el acceso a la fuente NVD. Guarde y cierre el archivo.

No copie el contenido de `.env` en capturas, documentos compartidos ni repositorios. No publique contraseñas ni tokens en tickets o logs.

15.3 Validar la configuración de Docker Compose

Desde la raíz del repositorio, ejecute:

```
docker compose config --quiet
```

El comando debe finalizar sin mensajes de error. Esto comprueba que la configuración YAML y las variables requeridas pueden resolverse.

Luego, construya las imágenes e inicie los servicios:

```
docker compose up -d --build
```

La primera ejecución descarga imágenes base y construye los componentes propios. Espere a que Docker termine el proceso antes de continuar.

15.4 Confirmar el estado de los servicios

Consulte el estado de todos los contenedores:

```
docker compose ps
```

El despliegue incluye los servicios de base de datos PostgreSQL, Redis, bootstrap de base de datos, inicialización de evidencias/logs/uploads, servidor de archivos, API, worker,

frontend, scanner, Prometheus, Alertmanager, Grafana, Loki y Alloy. Los servicios de inicialización pueden aparecer como terminados correctamente; los servicios de ejecución deben permanecer activos.

Revise los registros generales y los de la API si necesita confirmar el arranque:

docker compose logs --tail=100
docker compose logs --tail=100 securehub-api
docker compose logs --tail=100 securehub-db
docker compose logs --tail=100 securehub-worker

Para seguir los registros en tiempo real, use `docker compose logs -f securehub-api`. Pulse Ctrl+C para salir de la consulta de logs; esto no detiene el contenedor.

15.5 Abrir SecureHub y validar la API

Abra la interfaz en el navegador mediante la siguiente dirección:

http://localhost/

La API expone el endpoint de salud local en:

http://127.0.0.1:3000/health

La respuesta del endpoint confirma que la API responde. Después, inicie sesión en la interfaz y realice una comprobación funcional controlada, por ejemplo, consultar el panel principal o abrir un módulo disponible para la cuenta.

Si la página no carga, vuelva a `docker compose ps` y confirme que `securehub-frontend` y `securehub-api` están activos. Consulte los logs del servicio afectado.

15.6 Consultar las consolas de monitoreo

- Grafana: http://localhost:3001/ — abra el dashboard VulnTrack para revisar disponibilidad, peticiones HTTP por segundo, latencia y memoria de la API.
- Prometheus: http://localhost:9090/ — consulte métricas, reglas y estado de evaluación. En Status > Rule health se pueden revisar las reglas de alertas.
- Alertmanager: http://localhost:9093/ — consulte el estado de las alertas recibidas por el gestor.
- Los registros centralizados se consultan desde Grafana, utilizando la fuente de datos Loki y filtros por etiquetas/servicio. El servicio Loki no publica un puerto en el host en la configuración principal.

Use las consolas de monitoreo únicamente con acceso autorizado. La disponibilidad de una métrica depende de que el servicio emisor y la configuración de recopilación estén activos.

15.7 Operación cotidiana

- **Ver el estado actual:** docker compose ps
- **Consultar logs recientes:** docker compose logs --tail=100
- **Seguir logs de la API:** docker compose logs -f securehub-api
- **Reiniciar la API:** docker compose restart securehub-api
- **Reiniciar el conjunto de servicios:** docker compose restart

- **Iniciar los servicios existentes:** docker compose up -d
- **Actualizar/reconstruir después de cambios:** docker compose up -d --build
- **Detener los contenedores sin eliminar los datos persistentes:** docker compose down

No utilice `docker compose down -v` como método habitual de apagado: la opción `-v` elimina los volúmenes declarados por Compose y puede borrar la base de datos, evidencias, logs y datos de monitoreo.

15.8 Copia de seguridad de PostgreSQL

Realice la copia antes de actualizaciones importantes o de tareas administrativas sobre los datos. Obtenga los valores de `DB_USER` y `DB_NAME` del archivo `.env`, sin compartirlo. Desde PowerShell, ejecute el siguiente comando sustituyendo los marcadores:

docker compose exec -T securehub-db pg_dump -U <DB_USER> -d <DB_NAME> -Fc > .\securehub-backup.dump

Verifique que el archivo `securehub-backup.dump` exista y tenga un tamaño mayor que cero. Traslade una copia a una ubicación de respaldo protegida y con acceso restringido. No almacene la única copia dentro del mismo equipo donde reside el despliegue.

La restauración reemplaza datos y debe realizarse en una base de destino controlada. Para restaurar en un entorno de prueba, asegúrese de que el destino esté preparado y utilice `pg_restore` con las credenciales correspondientes. No restaure sobre una base con información que deba conservarse sin contar con una copia previa.

15.9 Solución de problemas habituales

- **Docker no responde o los comandos no conectan con el motor.** Abra Docker Desktop, espere a que el motor esté activo y vuelva a ejecutar `docker version` y `docker compose ps`.
- **La configuración no valida.** Ejecute `docker compose config` y revise nombres, sintaxis y valores del archivo `.env`. No comparta los valores secretos al solicitar ayuda.
- **Un servicio está reiniciándose o no aparece activo.** Consulte `docker compose logs --tail=100 <nombre-servicio>` y verifique sus dependencias. Corrija la causa indicada y reinicie únicamente el servicio afectado.
- **La interfaz no abre.** Compruebe que el puerto 80 esté libre y que `securehub-frontend` esté activo. Revise los logs del frontend y de la API.
- **La API no responde en /health.** Compruebe `securehub-api`, `securehub-db`, `securehub-redis` y `securehub-db-bootstrap`; revise los logs de API y base de datos.
- **No se ven métricas en Grafana.** Confirme que Prometheus esté activo, que la fuente de datos esté configurada y que el objetivo `vulntrack-api` esté siendo recopilado.
- **Falla una importación o carga de evidencias.** Revise los logs de API y del servidor de archivos y confirme que los servicios `securehub-files` y los volúmenes de datos estén activos.

15.10 Cierre controlado del entorno

Cuando termine la sesión de trabajo, ejecute desde la raíz del proyecto:

docker compose down

Este comando detiene y elimina los contenedores y la red de Compose, pero conserva los volúmenes nombrados. Para volver a iniciar el entorno, abra Docker Desktop y ejecute `docker compose up -d` desde la carpeta del proyecto.

16. Direcciones de acceso rápido

Componente	Dirección local
Interfaz SecureHub	http://localhost/
Salud de la API	http://127.0.0.1:3000/health
Grafana	http://localhost:3001/
Prometheus	http://localhost:9090/
Alertmanager	http://localhost:9093/

17. Criterios de verificación del despliegue

- El comando `docker compose config --quiet` finaliza sin errores.
- El comando `docker compose up -d --build` finaliza y crea las imágenes del proyecto.
- Los servicios de ejecución aparecen activos en `docker compose ps`; los servicios de inicialización terminan correctamente.
- La interfaz responde en `http://localhost/` y la API responde en `http://127.0.0.1:3000/health`.
- Grafana y Prometheus abren en sus puertos locales y muestran las métricas disponibles.
- Se ha ejecutado una operación funcional de prueba y se ha verificado que los datos persistentes se conservan al reiniciar los contenedores.